Lab 09 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — Scripts tegenover Compose

Antwoord. (1) **De toestand**: `up` vergelijkt het bestand met de bestaande containers en maakt alleen opnieuw aan wat veranderd is; een script herstart alles of faalt op "name already in use". (2) **De volgorde en het wachten**: `depends_on` + `healthcheck` laten wachten tot een dienst klaar is; een script rijgt `sleep`-commando's aan elkaar. (3) **De omvang**: `down` weet precies wat het project aangemaakt heeft (containers, netwerk, pod) en verwijdert dat zonder de rest aan te raken; een opruimscript vergeet altijd iets, of verwijdert te veel. Daar komen de samenvoeging van omgevingen (`-f`, override) en de leesbaarheid voor een nieuwkomer bij.

Waarom. Compose is **declaratief**: het beschrijft een doeltoestand en berekent het verschil. Een script is **imperatief**: het beschrijft handelingen, zonder het verwachte resultaat te kennen.

Nuance. Een script blijft nuttig voor wat Compose niet dekt (een secret van de engine aanmaken, een bestand initialiseren). De twee gaan samen: het script roept Compose aan.

Voorbeeld.

```
podman compose up -d && podman compose up -d     # de tweede keer: niets te doen, geen enkele container opnieuw aangemaakt
```

Vraag 2 — `.env` gaat de container niet in

Antwoord. `.env` wordt door **Compose** gelezen om de `${…}` in het YAML-bestand te substitueren (`${API_TAG:-1.0}`); het wordt aan geen enkele container doorgegeven. `APP_MESSAGE` hoorde in `environment:` van de dienst, of in een bestand waarnaar `env_file:` verwijst (`api.env`), dat wél aan de container doorgegeven wordt als `--env-file`.

Waarom. Twee mechanismen, twee momenten: de substitutie gebeurt bij het lezen van de YAML, de omgeving wordt opgebouwd bij het aanmaken van de container.

Nuance. Ook een variabele van de **shell** wordt voor de substitutie gebruikt, vóór `.env`. En `podman compose config` toont de gesubstitueerde YAML: staat `${APP_MESSAGE}` er niet in, dan heeft `.env` geen enkel effect.

Voorbeeld.

```
podman compose config | grep -c 'hallo vanuit dotenv'     # 0: de variabele wordt nergens gebruikt
podman compose exec api env | grep APP_MESSAGE             # de waarde uit api.env
```

Vraag 3 — Korte en lange `depends_on`

Antwoord. Korte vorm: `db` wordt **aangemaakt en gestart** vóór `api`, maar `api` vertrekt meteen; PostgreSQL aanvaardt nog geen verbindingen, Spring Boot faalt (`Connection refused`, `HikariPool … Exception during pool initialization`) en de container stopt met een foutcode — zonder `restart` blijft hij `Exited`. Lange vorm: `api` wordt pas gestart als de healthcheck van `db`

geslaagd is (`healthy`); Spring Boot vindt de database, alles is `Up`. De voorwaarde vereist een `healthcheck`: zonder heeft `db` geen gezondheidstoestand, en weigert podman-compose of wacht het eindeloos.

Waarom. "Gestart" is een toestand van de container; "klaar" is een toestand van de applicatie, die alleen een healthcheck kan waarnemen.

Nuance. Met `restart: unless-stopped` "werkt" de korte vorm uiteindelijk na enkele herstarts: dat is broos en verbergt het probleem. Onze nep-API verbindt niet met de database; het effect zie je aan de **volgorde** (`podman compose ps` toont `api` vijf seconden na `db` gestart), niet aan een mislukking.

Voorbeeld.

```
podman compose ps     # db  Up 11 seconds (healthy)  /  api  Up 6 seconds (healthy)  /  web  Up 1 second
```

Vraag 4 — `curl` ontbreekt in de image

Antwoord. `eclipse-temurin:21-jre-alpine` bevat geen `curl`: elke uitvoering van de healthcheck faalt (`exec: "curl": executable file not found`), en na `retries` mislukkingen wordt de toestand `unhealthy` — `web`, dat op `service_healthy` wacht, start nooit. Diagnose: `podman inspect stack_api_1 --format '{{json .State.Health.Log}}'`, of `podman healthcheck run stack_api_1`. Correctie: `wget` gebruiken (BusyBox, aanwezig in Alpine): `["CMD", "wget", "-qO-", "http://localhost:8080/actuator/health"]`.

Waarom. De healthcheck wordt **in** de container uitgevoerd, met de binaries van de image: een ontbrekend commando is een mislukking als een andere.

Nuance. Distroless-images hebben `curl` noch `wget` noch shell: de healthcheck moet dan een meegeleverd binary zijn (of, in Kubernetes, een HTTP-probe die de kubelet uitvoert, zonder commando).

Voorbeeld.

```
podman compose exec api wget -qO- http://localhost:8080/actuator/health     # {"status":"UP"}: het commando bestaat
podman compose exec api curl                                                # executable file not found
```

Vraag 5 — Named volume of bind mount voor de database

Antwoord. `pgdata:`: Compose maakt het volume `stack_pgdata` aan, Podman beheert de map, de bestanden zijn van het vertaalde UID van `postgres` (`100069` op WSL), `down` bewaart het, `down -v` verwijdert het. `./data`: jij maakt de map aan (of Podman maakt ze, als eigendom van jou), de bestanden erin hebben hetzelfde vertaalde UID, dus `ls ./data` antwoordt `Permission denied` en `git status` struikelt erover; `down -v` **raakt ze niet aan**. Voor de database: het named volume — te back-uppen met `podman volume export` (lab 06), buiten de repository, en zonder verrassingen met permissies.

Waarom. Compose beheert alleen wat het declareert; een hostpad is van jou, een volume is van hem.

Nuance. De bind mount is de juiste keuze voor **configuratie** (`nginx-proxy.conf`, `index.html`): geversioneerd, leesbaar, live aanpasbaar.

Voorbeeld.

```
podman compose down && podman volume ls | grep stack_pgdata     # nog altijd aanwezig
podman compose down -v && podman volume ls | grep -c stack_pgdata   # 0
```

Vraag 6 — Het spookproject

Antwoord. Zonder `name:` heet het project zoals de map: `stack` aan de ene kant, `stack-v2` aan de andere. De tweede `up` heeft `stack-v2_default`, `stack-v2_pgdata` (leeg) en `stack-v2_db_1` aangemaakt: een tweede stack, die het volume van de eerste niet ziet. Er is niets verloren: `podman volume ls` toont de twee volumes. Twee remedies: `name: stack` in het bestand, of `podman compose -p stack up -d` (of `COMPOSE_PROJECT_NAME`).

Waarom. De projectnaam is het voorvoegsel van alle resources; van naam veranderen is van wereld veranderen.

Nuance. Twee projecten naast elkaar zijn soms **gewild** (twee branches, twee versies) — op voorwaarde dat ze niet dezelfde poorten publiceren.

Voorbeeld.

```
podman volume ls --format '{{.Name}}'     # stack_pgdata  stack-v2_pgdata
podman compose -p stack-v2 down -v         # het spook verwijderen, het origineel bewaren
```

Vraag 7 — Twee antwoorden voor één container

Antwoord. `podman exec` zegt de waarheid: het toont de omgeving **van het proces**, `prod`. `podman compose exec` leest de **standaard** Compose-bestanden opnieuw (`compose.yaml` + `compose.override.yaml`, want er is geen `-f` opgegeven), leidt daaruit de omgeving van de dienst af en spuit die in het `exec`-commando: `dev` komt uit de override, niet uit de container. Om dat te vermijden: `exec` dezelfde `-f` geven als `up`, `COMPOSE_FILE=compose.yaml:compose.prod.yaml` in de shell zetten, of `podman exec` gebruiken.

Waarom. podman-compose geeft `--env` aan `podman exec` door om de gedeclareerde omgeving na te bootsen; het weet niet met welke bestanden de container aangemaakt is.

Nuance. Docker Compose gedraagt zich hetzelfde bij de keuze van de bestanden: `-f` wordt nooit onthouden. `COMPOSE_FILE` is de conventie van beide tools.

Voorbeeld.

```
podman compose -f compose.yaml -f compose.prod.yaml exec api env | grep APP_PROFILE   # prod
```

Vraag 8 — `restart: unless-stopped` zonder daemon

Antwoord. Bij Docker start de daemon samen met de machine en herstart hij de containers waarvan het beleid dat vraagt: de stack komt vanzelf terug. Bij rootless Podman is er **geen daemon**: na `wsl --shutdown` leest niemand het herstartbeleid; de containers blijven `Exited` tot een `podman compose up -d` of `podman start`. Het beleid werkt alleen **zolang Podman draait**

(crash van de applicatie → herstart). Voor een automatische start: **systemd**-units gegenereerd door Quadlet (lab 10), of `podman compose up -d` in een `systemd --user`-unit met *linger*.

Waarom. `--restart` wordt uitgevoerd door `conmon` en Podman voor een levende container; er bestaat geen permanent proces om bij het opstarten van het systeem in te grijpen.

Nuance. `podman generate systemd` (verouderd) en Quadlet produceren units die systemd bij het opstarten wél herstart — dat is het Podman-antwoord op `restart: always`.

Voorbeeld.

```
podman inspect stack_api_1 --format '{{.HostConfig.RestartPolicy.Name}}'   # unless-stopped
wsl --shutdown   # (vanuit Windows) en daarna: podman ps -a → Exited
```

Vraag 9 — Twee API's achter nginx

Antwoord. De twee containers dragen de **alias** `api` op het netwerk `stack_default`; de interne DNS geeft de twee adressen terug, en nginx (dat bij het opstarten resolvet) neemt wat het krijgt — in de praktijk een benaderende verdeling, geen echte load balancing. `--scale web=2` mislukt omdat de twee instanties allebei `18080` op de host willen publiceren: poortconflict. Met podman-compose kent `down` alleen instantie 1: je moet `stack_api_2` bij naam verwijderen (`podman rm -f -t 0 stack_api_2`) vóór `down`, anders kan `db` niet verwijderd worden (afhankelijkheid).

Waarom. De `scale` is een vermenigvuldiging van containers onder dezelfde dienstnaam; de echte verdeling is het werk van een load balancer (Kubernetes Service, lab 13).

Nuance. Docker Compose kan wel naar beneden schalen en bij `down` opruimen. Het is een van de concrete beperkingen van podman-compose die je moet kennen.

Voorbeeld.

```
podman run --rm --network stack_default alpine nslookup api 10.89.0.1 | grep Address   # twee adressen
podman rm -f -t 0 stack_api_2 && podman compose down -v
```

Vraag 10 — "name already in use"

Antwoord. Er bestaat al een `stack_db_1`, achtergelaten door een onvolledige `down` (bijvoorbeeld na een `--scale`, vraag 9) of door een handmatige `podman run --name`. `up` maakt een nieuwe container aan volgens het bestand en kan geen container "adopteren" die niet zijn projectlabels draagt of waarvan de configuratie verschilt. Herstel: `podman ps -a` om te kijken, `podman rm -f -t 0 stack_db_1` (het **volume** `stack_pgdata` wordt niet aangeraakt), en dan `up -d`.

Waarom. Containernamen zijn uniek per engine; Compose verwijdert nooit stilzwijgend een container die het niet herkent.

Nuance. Eerst `podman compose down` is de goede gewoonte; faalt die zelf, dan is opruimen bij naam de enige weg — nooit `podman rm -af` op een gedeelde machine.

Voorbeeld.

```
podman ps -a --filter name=stack_ --format '{{.Names}} {{.Status}}'
podman rm -f -t 0 stack_db_1 && podman compose up -d && podman volume ls | grep stack_pgdata
```

Vraag 11 — Secret via bestand

Antwoord. Met `secrets:` + `_FILE`: `podman compose config` toont `file: ./db_password.txt` (het pad), `podman inspect` toont `POSTGRES_PASSWORD_FILE=/run/secrets/db_password`, en Git ziet alleen het YAML-bestand — **als** `db_password.txt` in `.gitignore` staat. Met `environment: POSTGRES_PASSWORD: S3cr3t`: de waarde zit in de YAML (dus in Git), in `config`, in `inspect`. `db_password.txt`: buiten Git, rechten `600`, één per omgeving, of vervangen door een secret van de engine met `external: true` (vooraf `podman secret create`, lab 08).

Waarom. Compose mount het bestand in `/run/secrets/<naam>`; de waarde gaat door geen enkele metadata.

Nuance. Docker Compose buiten Swarm doet precies hetzelfde (een bestandsmount); `external: true` wijst daar naar een Swarm-secret, bij Podman naar een `podman secret`.

Voorbeeld.

```
podman compose config | grep -A1 'db_password:'          # file: ./db_password.txt
podman inspect stack_db_1 --format '{{json .Config.Env}}' | grep -c S3cr3t   # 0
```

Vraag 12 — Kubernetes-equivalenten

Antwoord. Dienst met `image` → **Deployment** (en zijn Pod); `ports` → **Service** (ClusterIP/NodePort) en, naar buiten, **Ingress**; volume `pgdata` → **PersistentVolumeClaim** (en StatefulSet voor een database); `healthcheck` → **readiness/liveness probes**; `depends_on` → **geen rechtstreeks equivalent**: Kubernetes beheert de startvolgorde niet, elke Pod probeert opnieuw tot zijn afhankelijkheden antwoorden, en de *readiness probe* voorkomt dat er verkeer gestuurd wordt voor het zover is; impliciet netwerk → het **cluster**-netwerk en de DNS van CoreDNS; `env_file` → **ConfigMap**; `secrets` → **Secret**.

Waarom. Kubernetes is ontworpen voor componenten die uitvallen en terugkomen: het vervangt "wachten" door "opnieuw proberen en niet routeren zolang het niet klaar is".

Nuance. *Init containers* kunnen op een afhankelijkheid wachten, maar dat is een omweg, niet het model.

Voorbeeld.

```
podman compose config > stack.yaml   # vertrekpunt voor kompose, of om de manifesten van lab 12 te schrijven
```

Vraag 13 — Compose in productie

Antwoord. Redelijk voor een interne applicatie met weinig verkeer, zonder beschikbaarheidseis, op een VM die een klein team beheert — wat heel wat echte applicaties beschrijft. Wat Compose niet biedt: (1) **hoge beschikbaarheid** (één VM, één single point of failure); (2) **updates zonder onderbreking** (een `up` maakt de container opnieuw aan, de dienst valt enkele seconden weg); (3) **spreiding over meerdere machines** en autoscaling; (4) **automatisch herstarten** in rootless zonder daemon, en versleuteld beheer van secrets. Het signaal: zodra je scripts schrijft om te doen wat Compose niet doet (failover, replicatie, rolling update), is het tijd voor een orchestrator.

Waarom. Compose is een tool voor **lokale compositie**, geen clusterscheduler.

Nuance. Tussen de twee bestaan eenvoudige oplossingen: Quadlet + systemd voor het herstarten, een load balancer voor twee Compose-VM's, of k3s op één node (lab 11), dat de Kubernetes-mechanismen brengt zonder de omvang van een cluster.

Voorbeeld.

```
podman compose -f compose.yaml -f compose.prod.yaml up -d && podman compose ps   # één VM, drie containers, één single point of failure
```